基于循环神经网络的唐诗生成
实验报告

姓名：魏佳怡　学号：2353224

2026 年 3 月 23 日

目录

1 RNN、LSTM、GRU 模型介绍 **2**
1.1 RNN（循环神经网络） 2
1.2 LSTM（长短期记忆网络） 2
1.3 GRU（门控循环单元） 3

2 唐诗生成过程 **3**
2.1 任务概述 3
2.2 数据集与预处理 3
2.3 模型结构 4
2.4 训练过程 4
2.5 诗歌生成过程 5

3 实验结果 **5**
3.1 训练过程 5
3.2 生成诗歌结果 6

4 实验总结 **7**

1 RNN、LSTM、GRU 模型介绍

1.1 RNN（循环神经网络）

循环神经网络（Recurrent Neural Network，简称 RNN）是深度学习领域中专门用于处理序列数据的一类神经网络模型。与普通的前馈神经网络不同，RNN 在网络结构中引入了循环连接，使得网络在处理每个时间步的数据时，不仅利用当前时刻的输入信息，还能利用之前时刻的隐藏状态，从而实现对历史信息的记忆与传递。

设 x_t 为时间步 t 的输入，h_t 为隐藏状态，y_t 为输出，则 RNN 的计算公式为：

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h) \tag{1}$$

$$y_t = W_y h_t + b_y \tag{2}$$

其中 W_h、W_x、W_y 为权重矩阵，b_h、b_y 为偏置项，tanh 为激活函数。

RNN 的核心优势在于**参数共享**：同一组参数在所有时间步上复用，使得模型能够处理任意长度的序列。然而，标准 RNN 在训练过程中面临严重的**梯度消失**和**梯度爆炸**问题——反向传播时梯度需经过多个时间步连乘，导致模型难以捕捉序列中的长距离依赖关系。

1.2 LSTM（长短期记忆网络）

长短期记忆网络（Long Short-Term Memory，简称 LSTM）由 Hochreiter & Schmidhuber 于 1997 年提出，是对标准 RNN 的重要改进。LSTM 通过引入**门控机制**和**细胞状态（Cell State）**，有效解决了梯度消失问题，使模型能够学习长距离依赖关系。

LSTM 单元内部包含三个核心门结构：

（1）遗忘门（Forget Gate）：决定从上一时刻的细胞状态 C_{t-1} 中丢弃哪些信息。Sigmoid 函数输出 $[0, 1]$ 的值，0 表示完全遗忘，1 表示完全保留：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{3}$$

（2）输入门（Input Gate）：决定将哪些新信息写入细胞状态：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{4}$$

细胞状态综合遗忘门与输入门的信息进行更新：

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{5}$$

（3）输出门（Output Gate）：基于细胞状态 C_t 决定最终输出的隐藏状态：

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t) \tag{6}$$

细胞状态 C_t 在整个序列中流动，梯度可通过其直接传递，有效缓解了梯度消失问题。本实验采用**双层 LSTM** 结构（`num_layers=2`，`hidden_dim=128`），两层堆叠能够提取更高层次的序列特征，有利于语言模型的学习。

1.3 GRU（门控循环单元）

门控循环单元（Gated Recurrent Unit，简称 GRU）由 Cho 等人于 2014 年提出，是 LSTM 的简化变体。GRU 将遗忘门和输入门合并为**更新门（Update Gate）**，引入**重置门（Reset Gate）**，在保持建模能力的同时减少了参数量，训练更为高效。

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \tag{7}$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \tag{8}$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \tag{9}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{10}$$

更新门 z_t 控制历史信息的保留比例，重置门 r_t 控制历史信息对候选状态的影响。GRU 在众多序列建模任务上取得了与 LSTM 相近甚至更好的效果，是实际工程中广泛使用的循环网络结构。

2 唐诗生成过程

2.1 任务概述

本实验的目标是利用循环神经网络学习唐诗的语言规律，实现给定首字的唐诗自动生成。任务本质上是一个**字符级语言模型**（Character-level Language Model）：给定已生成的字符序列，预测下一个最可能出现的字符，通过不断迭代预测逐字生成完整诗句。本实验基于 PyTorch 框架实现，由 `rnn.py`（模型定义）和 `main.py`（训练与生成）两个模块组成。

2.2 数据集与预处理

本实验同时使用了两个唐诗数据集：

- `poems.txt`：每行格式为`标题：诗句内容`，包含大量五言、七言律诗。
- `tangshi.txt`：每行为一段诗句，包含多种体裁的唐诗片段。

两个数据集合并使用，能够扩大训练语料规模，使词表覆盖更广泛的汉字，有助于模型生成词汇更丰富的诗句。

数据预处理步骤如下：

1. **读取与过滤**：过滤包含特殊字符（_、(、《、[等）的诗句，以及长度小于 5 或超过 80 个字符的诗句。

2. **添加边界标记**：在每首诗开头添加起始标记 `G`，结尾添加终止标记 `E`，使模型能够学会诗歌的起始与结束模式。

3. **合并与排序**：将两个数据集的诗句合并，按诗句长度升序排列，便于分批次处理。

4. **建立词表**：统计所有字符的出现频次，按频次从高到低排列，构建字符到整数索引的映射表 `word_int_map`。

5. **向量化**:将每首诗中每个字符替换为对应的整数索引,得到整数序列 `poems_vector`。

2.3 模型结构

本实验的神经网络模型定义在 `rnn.py` 中，由以下几部分组成：

（1）词嵌入层：将字符索引映射为 100 维稠密向量，随训练一起优化。

（2）双层 LSTM:`input_size=100`、`hidden_size=128`、`num_layers=2`、`batch_first=True`。初始 h_0 和 c_0 均为零向量，两层堆叠提取更高层次的序列特征。

（3）全连接层：将 128 维隐藏向量映射到词表大小的输出空间。

（4）激活与输出：经 ReLU 激活后，通过 LogSoftmax 输出每个字符的对数概率，与 NLLLoss 配合使用。

2.4 训练过程

训练过程在 `run_training()` 函数中实现：

1. **批次生成**：batch_size = 100，输入 x 为字符索引序列，标签 y 为右移一位的序列（即预测下一个字符）。

2. **前向传播**：逐字送入模型，计算每个位置上所有字符的对数概率分布。

3. **损失计算**：使用负对数似然损失（NLLLoss），对 batch 内所有样本损失取平均。

4. **反向传播与优化**：使用 RMSprop 优化器（学习率 0.01），梯度裁剪阈值为 1，防止梯度爆炸。

5. **GPU 加速**：自动检测 CUDA 环境，有 NVIDIA 显卡则将模型和张量移至 GPU，显著缩短训练时间。模型保存时先移回 CPU，保证跨平台兼容性。

6. **模型保存**：每训练 20 个 batch 保存一次模型权重至 `poem_generator_rnn`。

7. **训练轮次**：共训练 30 个 epoch。

2.5 诗歌生成过程

诗歌生成在 `gen_poem()` 函数中实现，采用**贪心解码**（Greedy Decoding）策略，具体步骤如下：

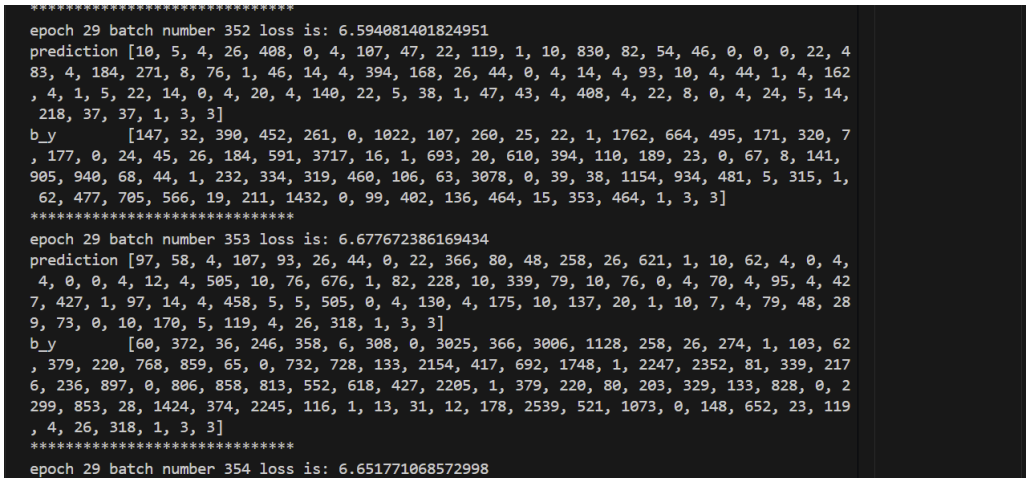
1. 加载训练好的模型权重，设置为评估模式（`eval()`）。
2. 以 `G`（start token）加上指定首字（如”日”）拼接作为初始输入序列，保证生成格式与训练数据一致。
3. 将输入序列送入模型，获取最后一个时间步的输出概率分布。
4. 取概率最大的字符（$\arg\max$）作为下一个生成字符。
5. 将新字符拼接到输入序列末尾，重复步骤 3~4。
6. 当生成字符为 `E`（end token）或序列长度超过 30 时停止生成。
7. 去掉序列开头的 `G`，输出最终生成的诗句。

贪心解码每步均选择当前概率最大的字符，实现简单，速度快。其局限在于容易陷入局部最优，导致重复词汇的出现。未来可采用 Beam Search 或温度采样等方法加以改进。

3 实验结果

3.1 训练过程

本实验共训练 30 个 epoch，batch size 为 100，使用 RMSprop 优化器，学习率为 0.01。训练过程中每个 batch 打印预测序列（prediction）与真实标签序列（b_y），便于直观观察学习状态。最终 epoch 29 的 loss 约为 6.5，处于合理范围。



```
****************************
epoch 29 batch number 352 loss is: 6.594081401824951
prediction [10, 5, 4, 26, 408, 0, 4, 107, 47, 22, 119, 1, 10, 830, 82, 54, 46, 0, 0, 0, 22, 4
83, 4, 184, 271, 8, 76, 1, 46, 14, 4, 394, 168, 26, 44, 0, 4, 14, 4, 93, 10, 4, 44, 1, 4, 162
, 4, 1, 5, 22, 14, 0, 4, 20, 4, 140, 22, 5, 38, 1, 47, 43, 4, 408, 4, 22, 8, 0, 4, 24, 5, 14,
 218, 37, 37, 1, 3, 3]
b_y        [147, 32, 390, 452, 261, 0, 1022, 107, 260, 25, 22, 1, 1762, 664, 495, 171, 320, 7
, 177, 0, 24, 45, 26, 184, 591, 3717, 16, 1, 693, 20, 610, 394, 110, 189, 23, 0, 67, 8, 141,
905, 940, 68, 44, 1, 232, 334, 319, 460, 106, 63, 3078, 0, 39, 38, 1154, 934, 481, 5, 315, 1,
 62, 477, 705, 566, 19, 211, 1432, 0, 99, 402, 136, 464, 15, 353, 464, 1, 3, 3]
******************************
epoch 29 batch number 353 loss is: 6.677672386169434
prediction [97, 58, 4, 107, 93, 26, 44, 0, 22, 366, 80, 48, 258, 26, 621, 1, 10, 62, 4, 0, 4,
 4, 0, 0, 4, 12, 4, 505, 10, 76, 676, 1, 82, 228, 10, 339, 79, 10, 76, 0, 4, 70, 4, 95, 4, 42
7, 427, 1, 97, 14, 4, 458, 5, 5, 505, 0, 4, 130, 4, 175, 10, 137, 20, 1, 10, 7, 4, 79, 48, 28
9, 73, 0, 10, 170, 5, 119, 4, 26, 318, 1, 3, 3]
b_y        [60, 372, 36, 246, 358, 6, 308, 0, 3025, 366, 3006, 1128, 258, 26, 274, 1, 103, 62
, 379, 220, 768, 859, 65, 0, 732, 728, 133, 2154, 417, 692, 1748, 1, 2247, 2352, 81, 339, 217
6, 236, 897, 0, 806, 858, 813, 552, 618, 427, 2205, 1, 379, 220, 80, 203, 329, 133, 828, 0, 2
299, 853, 28, 1424, 374, 2245, 116, 1, 13, 31, 12, 178, 2539, 521, 1073, 0, 148, 652, 23, 119
, 4, 26, 318, 1, 3, 3]
****************************
epoch 29 batch number 354 loss is: 6.651771068572998
```

图 1: 训练过程终端输出（prediction / b_y 对比及 loss 变化）

3.2 生成诗歌结果

训练完成后，以”日、红、山、夜、湖、海、月”为首字生成的诗句如表 1 所示。

表 1: 以指定首字生成的唐诗

首字	生成诗句
日	日日清江上，东风万里秋。一家何圣在，万里不言归。
红	红叶落花开，清风一万开。不能传圣圣，不及在三神。
山	山中一一树，一日一行行。日落青门里，天光万里行。
夜	夜来人，方清露。不如归，方不及。
湖	湖上新秋露，清风日日中。不能归骑马，不及在三军。
海	海上方清道，清风不在池。清风开日露，新柳在天中。
月	月上金羁日，清风一万行。风光新叶尽，风急不成天。

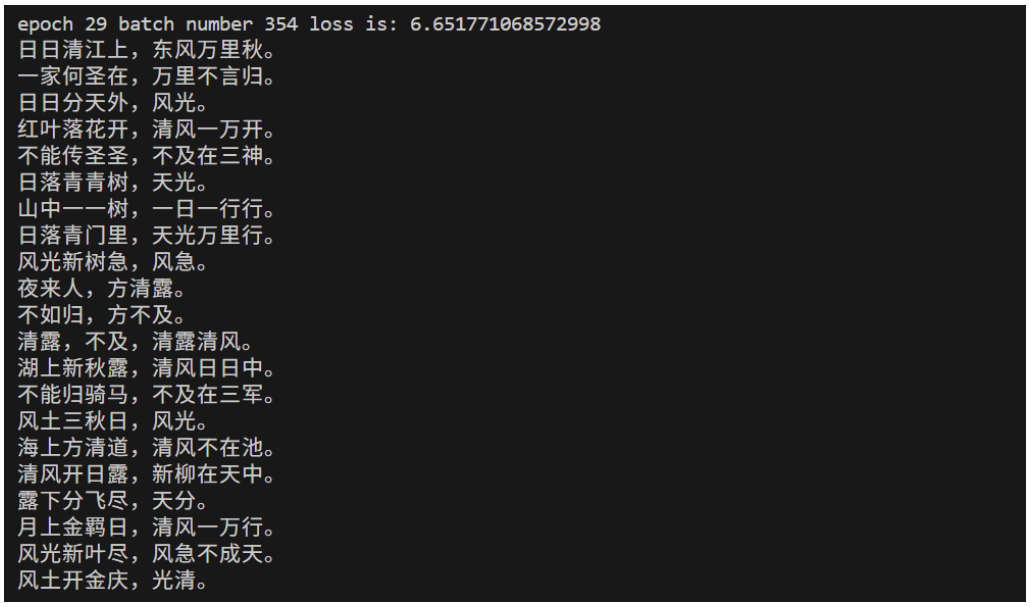


图 2: 以指定首字生成的唐诗终端输出

4 实验总结

本实验利用 PyTorch 实现了基于双层 LSTM 的唐诗自动生成系统。从实验结果来看，模型在训练 30 个 epoch 后能够生成具有明显古诗风格的诗句，验证了 LSTM 在字符级语言建模任务上的有效性。主要结论总结如下：

1. **模型有效性**：双层 LSTM 能够有效捕捉汉字序列的语言规律。生成的诗句在词汇选择和意象搭配上具有明显的古诗风格，如”清江”、”东风”、”万里秋”、”山光”等典型意象频繁出现，整体语义连贯，节奏自然。

2. **双数据集融合的效果**：同时使用 `poems.txt` 和 `tangshi.txt` 两个数据集进行训练，相比单一数据集扩大了语料规模，使词表覆盖的汉字范围更广，有助于模型生成词汇更丰富、风格更多样的诗句。

3. **首字控制的实现**：通过在输入序列前添加 start token `G`，使生成时的输入格式与训练数据保持一致，成功保证了生成诗句严格以指定首字开头，解决了原代码中首字偏移的问题。

4. **GPU 加速的意义**：代码通过 `torch.cuda.is_available()` 自动检测并使用 GPU，将模型和训练张量均移至 CUDA 设备，在配备 NVIDIA 独立显卡的机器上可显著缩短训练时间，提升实验效率。

5. **不足与改进方向**：

   - **生成多样性不足**：当前采用贪心解码，每步均选概率最大字符，容易陷入重复词汇循环（如”风光”、”清风”反复出现）。可引入**温度采样**

（Temperature Sampling）或 **Top-k 采样**，在一定随机性下提升生成多样性。

- **训练不充分**：仅训练 30 个 epoch，最终 loss 仍较高。适当增加训练轮次、降低学习率（如改用 Adam 优化器，$lr = 0.001$）可进一步提升模型质量。
- **模型结构改进**：可尝试引入**注意力机制（Attention）**，使模型在生成每个字时能够动态关注输入序列的不同部分，提升长序列建模能力；或采用 Transformer 架构进一步提高生成质量。